Sprint 2

Tecnologías de Procesamiento Big Data

3º Grado en Ingeniería Matemática e Inteligencia Artificial

Grupo 2:

Alejandro Alcázar Mendoza
Sergio Fernández Cordero
Liam Esgueva González

Índice

1. Introducción **3**
1.1. Contexto . . . 3
1.2. Descripción del problema y objetivos del sprint. . . . 3
1.3. Justificación . . . 3

2. Metodología **3**
2.1. Entorno de desarrollo . . . 3
2.2. Diseño de la solución . . . 4
2.3. Metodología SCRUM . . . 4

3. Resultados **5**
3.1. Descripción . . . 5
3.2. Ejecución . . . 6
3.3. Discusión de los resultados . . . 7

4. Conclusión **7**
4.1. Resumen del proceso . . . 7
4.2. Principales logros . . . 7

Sprint 2

1. Introducción

1.1. Contexto

El presente proyecto se desarrolla en el marco de la asignatura Tecnologías de Procesamiento Big Data y tiene como objetivo el diseño e implementación de una arquitectura Big Data completa orientada al análisis de datos de trading de criptomonedas. El proyecto simula un entorno real en el que distintas consultoras tecnológicas trabajan para un gran banco que desea crear un nuevo departamento especializado en el asesoramiento a clientes en trading de criptoactivos.

La solución propuesta se basa en tecnologías de Amazon Web Services (AWS) y contempla tanto el procesamiento de datos históricos en modo batch como la ingesta y análisis de datos en tiempo real (streaming). Para la gestión del proyecto se adopta la metodología ágil Scrum, estructurando el desarrollo en sprints incrementales y entregables funcionales al final de cada iteración.

Este documento recoge la planificación y los objetivos correspondientes al **segundo sprint del proyecto**.

1.2. Descripción del problema y objetivos del sprint.

El principal objetivo es realizar un gobierno de los datos almacenados en S3. Para ello, se debe:

- **Utilizar AWS Glue Data Catalog y AWS Glue Crawler** para catalogar (leer y guardar) los metadatos de los datos históricos almacenados en S3.
- Crear la **base de datos** con la nomenclatura `trade_data_<grupo>` (p. ej., `trade_data_imat3a01`).
- Implementar un *script* en Python que automatice la creación y la ejecución bajo demanda del **crawler**.

1.3. Justificación

Este *script* es clave porque permite ejecutar el *crawler* de Glue de forma repetible y bajo demanda, garantizando que el catálogo de metadatos esté siempre actualizado con los datos más recientes en S3. Además, reduce errores manuales (nombres de base de datos y tablas) y facilita el gobierno del dato al estandarizar el proceso de catalogación y su trazabilidad.

2. Metodología

2.1. Entorno de desarrollo

El inicio de este sprint se centró en adaptar la organización del almacenamiento en Amazon S3, modificando la estructura definida en el sprint anterior a la nueva estructura requerida para el escenario actual. En el Sprint 1 se trabajaba con un *bucket* independiente por cada año, incluyendo en su interior una carpeta `month` y, dentro de ella, su fichero `.csv` correspondiente.

Para este sprint se unificó el almacenamiento en un único *bucket* y se reorganizó el contenido siguiendo un esquema más adecuado para su catalogación automática: se creó una carpeta `ada` (equivalente a una capa de *raw data* de la criptomoneda) y, dentro de esta, una jerarquía `year/month/archivo.csv`. Este cambio se implementó modificando las clases `S3YearlyBucketManager`, responsable de la creación del bucket y su estructura interna, y `CryptoDataLoader`, encargada de introducir los ficheros `.csv` en el *bucket* y ruta correspondiente.

El propósito de esta transformación fue facilitar el uso de *AWS Glue Crawler*: al trabajar con múltiples *buckets*, el crawler no permite de forma sencilla construir una única *database* a partir de fuentes separadas. Al unir los datos en un único *bucket* con una estructura de particionado clara, se permite que el crawler obtenga y catalogue automáticamente los datos.

2.2. Diseño de la solución

Con el objetivo de automatizar la catalogación de los datos almacenados en S3, se incorporó al proyecto un nuevo módulo en el directorio `glue/`, compuesto por un fichero `__init__.py` y el *script* `glue_pipe.py`. En este último se definen, junto con funciones auxiliares de normalización y nomenclatura, dos clases principales: `GlueDatabaseManager` y `GlueCrawlerManager`.

En primer lugar, la configuración necesaria para operar con AWS Glue se centraliza en el *dataclass* `GlueConfig`, que encapsula parámetros como la región, el identificador del grupo, el nombre del bucket y el `ARN` del rol de Glue. A partir de esta configuración, `GlueDatabaseManager` se encarga de crear (o verificar la existencia) de la base de datos del *Glue Data Catalog*, con nombre `trade_data_<group>`. Su método `ensure_database` implementa un comportamiento funcional en todo caso: consulta la base de datos y, si no existe, la crea con una descripción asociada.

Por su parte, `GlueCrawlerManager` gestiona el *crawler* que incorpora al catálogo los datos del bucket. La clase construye el objetivo de rastreo como `s3://<bucket>/<crypto>/` y asegura que las tablas generadas se almacenen en la base de datos anterior. El método `ensure_crawler` también asegura que funcione en cualquier caso: si el crawler existe, lo actualiza para alinear el `target`, el prefijo de tablas y la base de datos; y si no existe, lo crea. Adicionalmente, se fuerza que el `TablePrefix` incluya el identificador de la criptomoneda para mantener una nomenclatura consistente.

Para que el crawler pueda acceder a los datos, es imprescindible asignarle un rol (`glue_role_arn`) con permisos suficientes para leer en S3 y escribir metadatos en el catálogo de Glue.

2.3. Metodología SCRUM

Para seguir la metodología SCRUM, aunque el sprint fuese únicamente de dos semanas, antes de comenzar se analizó la historia de usuario correspondiente al sprint actual:

Historia de Usuario (HU-3)

3.1. Como:
Manager del proyecto.

3.2. Quiero:
Que las consultoras:

- Utilicen el servicio AWS Glue Data Catalog y AWS Glue Crawler para leer y guardar los metadatos relacionados con los datos históricos almacenados en S3.
- El nombre de la base de datos debe tener la siguiente nomenclatura:
  - `trade_data_<grupo>` (ej.: `trade_data_imat3a01`).
- El nombre de las tablas debe contener en su nombre referencias a la criptomoneda:
  - Ejemplo: `btc`.
- Se deberá hacer un *script* en Python para automatizar la creación y ejecución a demanda del crawler.

3.3. Para:
Realizar un gobierno de los datos almacenados en S3.

A partir de la Historia de Usuario se ha realizado un análisis previo para organizar el trabajo del sprint en tareas concretas, asignar responsabilidades dentro del equipo y establecer los criterios de validación necesarios para garantizar que el resultado cumple con los objetivos planteados.

Descomposición en tareas

Tareas del Sprint

ID	Tarea	Asignado a
#18	Documentation	Sergio y Alejandro
#16	Create crawler script	Sergio y Alejandro
#15	Adaptation: data uploader sprint1 to sprint2	Sergio
#14	Adaptation: bucket manager sprint1 to sprint2	Alejandro
#13	Pipeline integration	Alejandro
#11	Upload data into buckets	Sergio

3. Resultados

3.1. Descripción

Para comprobar el correcto funcionamiento completo del gobierno del dato, se añadió en `glue_pipe.py` la clase `AdaquantGlueCatalogOrchestrator`. Esta clase actúa como orquestador del proceso completo: inicializa la *database* del *Glue Data Catalog* (mediante `GlueDatabaseManager`), crea o actualiza el crawler asociado (mediante `GlueCrawlerManager`) y automatiza la carga de datos desde el *bucket* de S3 hacia la base de datos del catálogo.

Una vez aplicados los cambios de estructura heredados del sprint anterior y verificado que el acceso al bucket era correcto, el orquestador se invocó desde el fichero `test_glue.py` como prueba de

integración. De esta forma se validó que la creación del catálogo y la ejecución del crawler funcionan correctamente, cumpliendo completo del sprint.

3.2. Ejecución

A continuación se muestran capturas del proceso de creación/uso del catálogo, la ejecución del *crawler* y el resultado en forma de tablas generadas.

```
(adaquant_venv) aleja@HPAlex:~/adaquant/ADAQuant$ python test_glue.py
Creating Glue database: trade_data_imat3a02
Crawler already exists: crawler_trade_data_imat3a02_bronze_ada. Updating targets/prefix just in case.
Started crawler: crawler_trade_data_imat3a02_bronze_ada
Crawler state changed: None -> RUNNING
Crawler state changed: RUNNING -> STOPPING
Crawler state changed: STOPPING -> READY
Crawler finished successfully: crawler_trade_data_imat3a02_bronze_ada
Crawler used: crawler_trade_data_imat3a02_bronze_ada
```

Figura 1. Ejecución del script de automatización del crawler (`test_glue.py`).

#	symbol	open	high	low	close	volume	day	year	month
1	BINANCE:ADAUSDT	0.3518	0.3644	0.3493	0.3601	6.767179769999991E7	1	2023	03
2	BINANCE:ADAUSDT	0.3601	0.3615	0.3445	0.3504	8.185457680000006E7	2	2023	03
3	BINANCE:ADAUSDT	0.3505	0.3509	0.3212	0.3431	1.630259220000001E8	3	2023	03
4	BINANCE:ADAUSDT	0.3431	0.3448	0.3303	0.3366	5.64327719E7	4	2023	03
5	BINANCE:ADAUSDT	0.3366	0.343	0.3346	0.337	5.326937070000005E7	5	2023	03
6	BINANCE:ADAUSDT	0.337	0.3387	0.3291	0.3305	7.288219720000006E7	6	2023	03
7	BINANCE:ADAUSDT	0.3306	0.3348	0.3209	0.3306	9.920965450000004E7	7	2023	03
8	BINANCE:ADAUSDT	0.3305	0.3328	0.3133	0.3175	1.1050110049999988E8	8	2023	03
9	BINANCE:ADAUSDT	0.3175	0.3223	0.3012	0.3101	1.915200241999999E8	9	2023	03
10	BINANCE:ADAUSDT	0.3102	0.3182	0.2976	0.3149	1.8444662920000014E8	10	2023	03

Figura 2. Creación de la base de datos en AWS Glue Data Catalog.

Schema (9)

View and manage the table schema.

Edit schema as JSON | Edit schema

Filter schemas

1

#	Column name	Data type	Partition key	Comment
1	symbol	string	-	-
2	open	double	-	-
3	high	double	-	-
4	low	double	-	-
5	close	double	-	-
6	volume	double	-	-
7	day	bigint	-	-
8	year	string	Partition (0)	-
9	month	string	Partition (1)	-

Figura 3. Tablas generadas en el catálogo tras la ejecución del crawler.

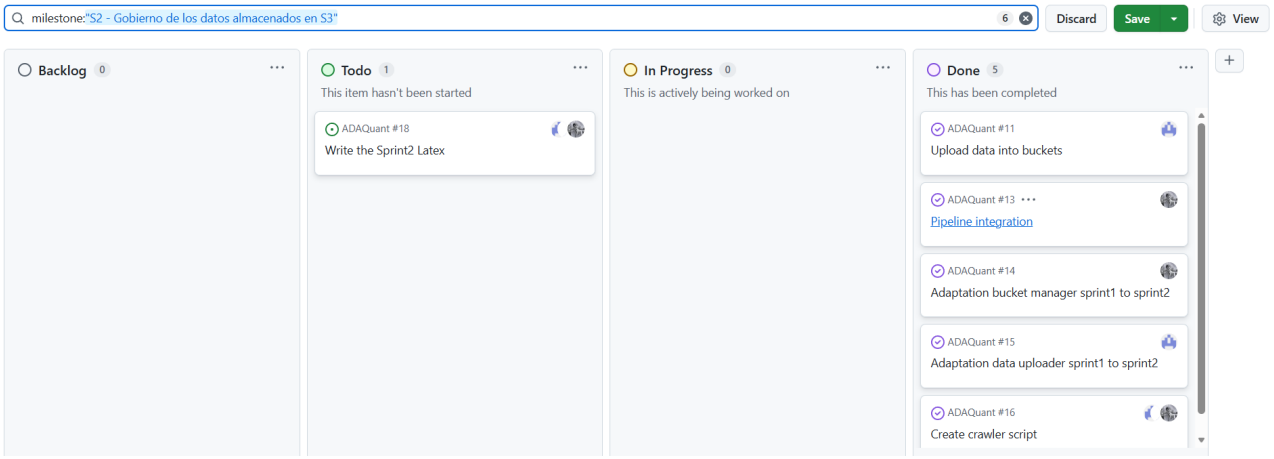


Figura 4. Milestone

3.3. Discusión de los resultados

A la vista de la estructura actual de carpetas y de la organización modular del código, se ha decidido mantener una separación clara de responsabilidades entre componentes. Este enfoque facilita la evolución del sistema y permitirá adaptar lo implementado en este sprint a los requisitos y cambios previstos para el Sprint 3 realizando la menor cantidad de cambios.

Además, se mantendrá la dinámica de trabajo mediante la documentación continua del progreso, registrando decisiones y tareas a través de *milestones* e *issues* en el repositorio de GitHub.

4. Conclusión

4.1. Resumen del proceso

Durante este sprint se reorganizó el almacenamiento de históricos en S3 para facilitar su catalogación y se incorporó un flujo reproducible de gobierno del dato basado en AWS Glue. A partir de la definición de la base de datos y el crawler, se automatizó su creación/actualización y ejecución bajo demanda mediante un *script* en Python, validando el proceso con una prueba de integración y verificando la generación de las tablas en el Data Catalog.

4.2. Principales logros

- Adaptación de los componentes del Sprint 1 (estructura en S3, *bucket manager* y *data uploader*) al nuevo esquema de almacenamiento y particionado requerido en el Sprint 2.
- Implementación de un flujo de gobierno del dato con AWS Glue (Data Catalog + Crawler), automatizando su creación/actualización y ejecución bajo demanda mediante un *script* en Python.